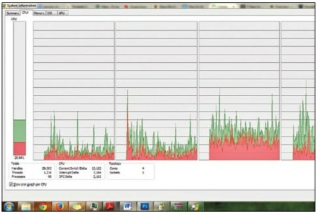

Figure 9: Visualising the CPU process

written and read between differing processes).

## Visualising the process using Process Explorer

Process Explorer helps to visualise the process which, in turn, helps to deeply observe the process and its handles. It can be downloaded from *https://technet.microsoft.com/en-in/sysinternals/bb896653.aspx*

The tools explained above can help you to learn and analyse the behaviour of a process/thread. The process has to be carried out manually. END

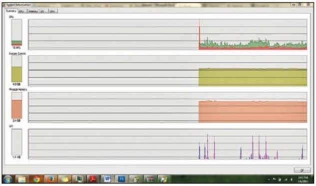

Figure 10: A visual summary of all the processes

### References

[1] *https://technet.microsoft.com/enin/sysinternals/bb896653.aspx*
[2] *https://download.sysinternals.com/files/SysinternalsSuite.zip*

**By: Sibi Chakkaravarthy**

The author holds an M. Tech degree in computer science and engineering and is working on a PhD at Anna University. Currently, he is with the AU-KBC Research Centre, MIT, Chennai. His research interests include network security, wireless security, wireless sensor networks, cloud security, etc. He can be reached at *sb(DOT)sibi(AT)gmail(DOT)com.*

***Continued from page 46...***

From the output of the above command, you can get the RUN_ID of the client and run the *show* command to get client details. For example:

```
# knife runs show c40csacd2-6778-122d-45589-3e1c4e55f741
```

Execution of the above command will display the running information of the client like node name, ID, status, run resources, checksum, permissions, user and other relevant details. With this step, the client is successfully installed, configured and able to communicate with the registered server.

## Preparing the reporting facility

Now the client is completely ready, and we need to enable its reporting facility so that reporting data is collected in the server and displayed in the management console. This can be done by editing the *client.rb* file and setting the option *enable_reporting true.* At the same time, run the reporting service in Chef Server (Guest OS) as follows:

```
#opscode-reporting-ctl start
```

To get some sample reporting scripts written in Ruby, you can download them from the Git repository *https://github.com/Chef-cookbooks/Chef-reporting.*

Use the *git fetch* command to download these Ruby scripts and add *Chef-reporting::default* to *client.rb* to enable the publishing of reporting data. There's no problem in introducing Ruby scripts suddenly.

So far, we have looked at how to set up a client VM to install the Chef client, and how to install a Chef client and Chef reporting service. Also, we have learnt the purpose of the *knife* tool, as well as how to install and configure it. In our next article, we will discuss Ruby scripts and their implications in Chef Cookbook preparation. Also, we will cook up some recipes for Chef configuration, and learn how to do a typical deployment of the application and how Knife is used in the Chef environment.

In subsequent articles in this series, we want to showcase a typical real-time banking/financial infrastructure to show how Chef helps in automating the deployment process with just a single button click, as well as how it helps in resilience and automating scaling of infrastructure to meet customer demands. END

**By: Magesh Kasthuri and Dr B. Thangaraju**

Kasthuri is a senior distinguished member of the technical staff at Wipro. He is a senior technical consultant in the Java-Technology Practices Group and can be reached at *magesh.kasthuri@wipro.com.*

Thangaraju is an open source software evangelist, who works at Talent Transformation, Wipro Technologies, Bangalore and can be reached at *balat.raju@wipro.com.*